WUNK 202

INTRODUCTION TO WUNKEMATICS

LECTURES DELIVERED BY
JONATHAN BINGUS

UNIVERSITY OF IPSUM
FALL 2026

JOHN STUDENT

The Pond

Chapter 1

This is a tremendously inspirational quote that sets the tone of this course; truly, one of the epigraphs of all time.

— Jonathan Bingus

1.1. Introduction

We begin our study of wunk analysis by investigating the pond. The pond is a central structure in applied wunkebra, because of its use in telecommunications and biology. Intuitively, a pond can be compared to a body of water (as in the usual sense of the word); it comprises a liquid medium, and it may contain objects within the medium.

Definition 1.1. A *pond* is a set of wunks P along with a *medium element* M that satisfies the following properties (pond axioms):

(1) For all distinct wunks $w_1, w_2 \in P$ such that both w_1 and w_2 are fish, if w_1 is dancing, then w_2 is not dancing.[1] (Fish axiom)
(2) For each wunk $w \in P$, there exists an anti-wunk $\overline{w} \in P$ such that the combination of w and $\overline{w}$ results in annihilation, i.e. $w\overline{w} = M$. (Anti-wunk axiom)

The most commonly used pond is $\mathbb{P}_1$, where the medium M is water, and the wunks are acidic (A) and basic (B) fish:

$$A = \{a_1, a_2, ...\}, \quad B = \{b_1, b_2, ...\},$$
$$\mathbb{P}_1 = A \cup B.$$

Proposition 1.2. *The set $\mathbb{P}_1$ is a pond.*

Proof. We show that $\mathbb{P}_1$ satisfies the pond axioms.

(1) This part of the proof has been left as an exercise to the reader.
(2) For acidic wunks, the basic wunk is the anti-wunk, and vice-versa. The combination of an acidic and basic wunk produces water, which is by definition the medium of $\mathbb{P}_1$. Therefore, $\mathbb{P}_1$ is a pond. ∎

1.2. WunkPy

The WunkPy library provides many convenient utilities for working with wunks. See Listing 1 for a usage example.

[1] Informally, this axiom is often stated as "two fish may not dance in the same pond."

```
from wunkpy import Pond, Wunk

p = Pond()
for w in (Wunk(dancing=True), Wunk(dancing=False)):
  p.add(w)
```

LISTING 1: Initializing a pond in WunkPy

1.3. *Examples*

EXAMPLE 1.1. Suppose we construct a pond chain of length $n \in \mathbb{N}$, where each pond is isomorphic to $\mathbb{P}_1$. Alice (at pond 1) makes a fish other than f_1 dance. What does Bob (observing pond n) see with his fish? Notably, does fish f_n annihilate or stop dancing?

Solution. We examine the cases where n is even, and n is odd. Using proof by I said so, the statement holds. ∎

Guide to Mousse

Chapter 2

2.1. Introduction

Mousse is a template intended for writing lecture notes, specifically intended for use in STEM courses. Each Typst file is supposed to represent a complete textbook for an individual course. Mousse's design is inspired by old-ish math books. The name of the template itself is just a random French word, because French sounds fancy. For up-to-date information, see the source code of this template at https://github.com/dogeystamp/mousse-notes.

Mousse is intended to be batteries-included, and provides tools you might need to write notes, e.g. Theorem and Example environments. This chapter shows by example how to use the functions provided in Mousse. Please reference the source code of this document while reading to see how the functions are used.

2.2. Document Structure

In Mousse, first level headings (=) represent chapters. Second and third level headings (==, ===) are sections and subsections. As always, you can reference sections and chapters using normal Typst methods: Section 2.2, Chapter 2.

2.2.1. Subsection

This is what a subsection looks like.

2.2.1a. Subsubsection

And a subsubsection.

2.3. Math Equations

Math equations look like this:

$$1 + 1 = 2$$

When you add a label to an equation, it gains a number:

$$1 + 1 = 2 \tag{1}$$

You can then reference the equation with the label, e.g. see (1).

2.4. Indent control

Due to limitations with Typst,[1] you can not break a paragraph after a display math equation or any other block element. That is, after a block element, there will never be an indent. Mousse provides a workaround for this which lets you add an indent. This feature applies to display math, theorem environments, and figures.

[1] See: https://github.com/typst/typst/issues/3206

$$1 + 1 = 2$$

If you add a blank line after an equation, it will give an indent to the next paragraph.

$$1 + 1 = 2$$

If you don't, it will consider the following text to be part of the same paragraph, so no indent will be added.

The method used to provide this feature is hacky, and it may break in future releases of the Typst compiler or no longer be necessary. A notable limitation is that it can't recurse into containers like `#block`, or even `#set` and `#show`. Because of this, you must keep the `#show: style` rule after any other `#set` or `#show` rules, otherwise it will break.

2.5. Theorem Environments

MOUSSE provides the `theorem()`, `proposition()`, `lemma()`, `corollary()`, `definition()`, `example()`, `solution()`, `proof()` and `remark()` environments by default. You can also create your own; see the source code in `src/_theorems.typ` to see how to do that.

Here are some examples of using theorems. You can make unnamed theorems:

THEOREM 2.1. *For all $x \in \mathbb{R}$, we have something.*

You can set a name:

THEOREM 2.2 (Pythagorean). *Bla bla bla $a^2 + b^2 = c^2$.*

You can reference theorems (see Theorem 2.3):

THEOREM 2.3. *For all $x \in \mathbb{C}$, we have something.*

For proofs, you must add `$qed$` by yourself. In MOUSSE, `$qed$` is intended to be next the the content, rather than at the end of the line. For example:

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat. ∎

2.6. Further Configuration

MOUSSE is an opinionated template, and offers no configuration options. The recommended way for you to change the style of the template is to fork the repository. (If you make an improvement that can benefit all users of this template, please consider making a PR.)

The Typst compiler does not usually have large breaking changes, so you should be able to use your fork indefinitely without having to backport changes from the upstream MOUSSE package.